

Web Service Design and Practice

12. 테스트와 운영

Dr. Kyudong Park

School of Information Convergence



Contents

- [서비스 운영을 위한 Packages](#)
- 부하 테스트
- 사용자 행동 분석

1. 환경 변수에 대한 이해

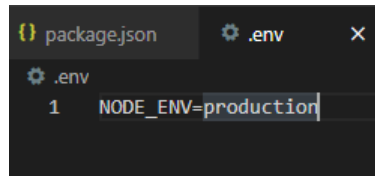
- 어느 환경에 배포하느냐에 따라 다르게 설정해야하는 항목은 환경 변수를 통해 관리
 - 개발 환경에서는 로컬 DB를 사용해야하는데, 운영 환경에서는 원격 DB를 사용해야하는 경우
 - DB password나 API key와 같은 인증 정보는 공개된 코드 저장소에 올리면 안 되기 때문에 환경 변수로 저장해놓고 사용
- node.js에서는 process.env (내장객체) 를 통해 환경 변수에 접근 가능

```
$ node
Welcome to Node.js v14.15.0.
Type ".help" for more information.
> process.env.USER
'dale'
> process.env.HOME
'/Users/dale'
> process.env.LANG
'en_US.UTF-8'
> process.env.API_KEY
undefined
```

```
$ API_KEY=abc DB_PASSWORD=1234 node
Welcome to Node.js v14.15.0.
Type ".help" for more information.
> process.env.API_KEY
'abc'
> process.env.DB_PASSWORD
'1234'
```

1. 환경 변수에 대한 이해

- dotenv 패키지
 - root의 .env 파일을 만들어서 환경 변수로 관리 가능
- 주의사항
 - .gitignore 에 .env 를 반드시 등록
 - 그렇지 않으면?



```
1 var express = require('express');
2 var router = express.Router();
3 require('dotenv').config();
4
5 /* GET home page. */
6 router.get('/', function(req, res, next) {
7   //res.render('index', { title: 'Express' });
8   console.log(process.env.NODE_ENV);
9   res.status(200).json(
```

2. morgan

- 로그를 쉽게 남길 수 있도록 도와주는 미들웨어
- 개발용으로 설정된 익스프레스 미들웨어를 배포용으로 전환
 - process.env.NODE_ENV는 배포 환경인지 개발 환경인지를 판단할 수 있는 환경 변수
 - app.use(morgan('dev'));
 - method, 접속 주소, 상태(status), 걸린 시간, 용량 순으로 로그를 띄워줌 (주로 개발 시 활용)
 - app.use(morgan('combined'));
 - 브라우저 및 시간, ip, 환경까지 더 자세한 로그를 띄워줌 (주로 배포 시 활용)

```
if (process.env.NODE_ENV === 'production') {  
  app.use(morgan('combined'));  
} else {  
  app.use(morgan('dev'));  
}
```

```
[nodemon] starting `node app.js`  
서버 실행  
GET /category/java 200 4.637 ms - 10  
[nodemon] restarting due to changes...  
[nodemon] starting `node app.js`  
서버 실행  
::1 - - [01/Apr/2021:11:26:26 +0000] "GET /category/java HTTP/1.1" 304 - "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.114 Safari/537.36"
```

3. express-session

- 설정들을 배포용과 개발용으로 분기 처리
 - production일 때는 proxy를 true, secure를 true로
 - 단, https를 적용할 경우에만 secure를 true로 하고, 노드 앞에 다른 서버를 두었을 때 proxy를 true로 함

```
const sessionOption = {
  resave: false,
  saveUninitialized: false,
  secret: process.env.COOKIE_SECRET,
  cookie: {
    httpOnly: true,
    secure: false,
  },
};
if (process.env.NODE_ENV === 'production') {
  sessionOption.proxy = true;
  // sessionOption.cookie.secure = true;
}
app.use(session(sessionOption));
```

```
app.get('/login', (req, res) => {
  sess = req.session;
  sess.username = "velopert"
});

app.get('/', (req, res) => {
  sess = req.session;
  console.log(sess.username);
});
```

4. cross-env

- 동적으로 process.env 변경 가능
 - package.json을 다음과 같이 수정(배포용과 개발용 스크립트 구분)

```
package.json
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "server.js",
  "scripts": {
    "start": "NODE_ENV=production PORT=80 node server",
    "dev": "nodemon server",
    "test": "jest"
  },
  ...
}
```

- 문제점: 윈도우에서는 NODE_ENV를 위와 같이 설정할 수 없음
- 이 때 cross-env가 필요

콘솔

```
$ npm start
```

'NODE_ENV'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는 배치 파일이 아닙니다.

4. cross-env

- cross-env 설치 후 적용

콘솔

```
$ npm i cross-env
```

- package.json도 수정

package.json

```
{  
  ...  
  "scripts": {  
    "start": "cross-env NODE_ENV=production PORT=80 node server",  
  },  
}
```


5. sanitize-html

- XSS(Cross Site Scripting) 공격 방어
 - 웹 애플리케이션에서 많이 나타나는 취약점의 하나로 웹사이트 관리자가 아닌 사용자가 웹페이지에 악성 스크립트를 삽입할수 있는 취약점
 - 허용하지 않은 html 입력을 막음
 - npm i sanitize-html
 - 아래처럼 빈 문자열로 치환됨

```
const sanitizeHtml = require('sanitize-html');

const html = "<script>location.href = 'https://gilbut.co.kr'</script>";
console.log(sanitizeHtml(html)); // ''
```

6. csrf

- CSRF(Cross Site Request Forgery) 공격 방어
 - 공격자가 승인되지 않은 도메인의 요청을 백엔드로 보내 악의적인 작업을 수행하는 공격
 - npm i csrf
 - csrfToken을 생성해서 프론트로 보내주고(쿠키로) Form 등록 시 csrfToken을 같이 받아 일치하는지 비교

```
const csrf = require('csrf');
const csrfProtection = csrf({ cookie: true });

app.get('/form', csrfProtection, (req, res) => {
  res.render('csrf', { csrfToken: req.csrfToken() });
});

app.post('/form', csrfProtection, (req, res) => {
  res.send('ok');
});
```

7. pm2

- 원활한 서버 운영을 위한 패키지
 - 서버가 에러로 인해 꺼졌을 때 서버를 다시 켜 줌
 - 멀티 프로세싱 지원(노드 프로세스 수를 1개 이상으로 늘릴 수 있음)
 - 요청을 프로세스들에 고르게 분배
 - 단점: 프로세스간 서버의 메모리 같은 자원 공유 불가
 - 극복: memcached나 redis같은 메모리 DB 사용(공유 메모리를 별도 DB에 저장)

7. pm2

- pm2 전역 설치 후, 명령어 사용
 - npm i pm2 -g
 - package.json 스크립트 수정

```
"scripts": {  
  "start": "cross-env NODE_ENV=production PORT=80 pm2 start server.js",  
}
```

- pm2 start 파일명으로 실행

콘솔

```
$ npm start
```

♥ 그림 15-1 npm start 명령 실행 화면

```
[PM2] Spawning PM2 daemon with pm2_home=C:\Users\speak\.pm2  
[PM2] PM2 Successfully daemonized  
[PM2] Starting C:\Users\speak\WebstormProjects\nodejs-book\ch15\15.1\nodebird\server.js in fork_mode (1 instance)  
[PM2] Done.
```

id	name	namespace	version	mode	pid	uptime	cpu	status	cpu	mem	user	watching
0	server	default	0.0.1	fork	34692	0s	0	online	0%	34.9mb	zerocho	disabled

7. pm2

- pm2 list로 프로세스 목록 확인 가능
 - 프로세스가 백그라운드로 돌아가기 때문에 콘솔에 다른 명령어 입력 가능

콘솔

```
$ npx pm2 list
```

♥ 그림 15-2 npx pm2 list 명령 실행 화면

id	name	namespace	version	mode	pid	uptime	↻	status	cpu	mem	user	watching
0	server	default	0.0.1	fork	34692	3m	0	online	0%	45.2mb	zerocho	disabled

Note ≡

리눅스나 맥에서 pm2 실행 시

리눅스나 맥에서 pm2 실행 시 1024번 이하의 포트를 사용하려면 관리자 권한이 필요합니다. 따라서 sudo를 명령어 앞에 붙여 실행합니다. 앞으로 나오는 다른 명령어도 sudo npm start, sudo pm2 list, sudo pm2 kill, sudo pm2 monit처럼 하면 됩니다.

7. pm2

- pm2 start [파일명] -i [프로세스 수] 명령어로 멀티 프로세싱 가능
 - 프로세스 수에 원하는 프로세스의 수 입력
 - 0이면 CPU 코어 개수만큼 생성, -1이면 CPU 코어 개수보다 1개 적게 생성
 - -1은 하나의 프로세스를 노드 외의 작업 수행을 위해 풀어주는 것

package.json

```
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "server.js",
  "scripts": {
    "start": "cross-env NODE_ENV=production PORT=80 pm2 start server.js -i 0",
    "dev": "nodemon server",
    "test": "jest"
  },
  ...
}
```

7. pm2

- pm2 monit으로 프로세스 모니터링

콘솔

```
$ npx pm2 monit
```

- 프로세스별로 로그를 실시간으로 볼 수 있음

▼ 그림 15-5 npx pm2 monit 명령 실행 화면

Process List				server Logs	
[0]	server	Mem: 46 MB	CPU: 0 %		
[1]	server	Mem: 46 MB	CPU: 0 %		
[2]	server	Mem: 46 MB	CPU: 0 %		
[3]	server	Mem: 46 MB	CPU: 0 %		
[4]	server	Mem: 48 MB	CPU: 0 %		
[5]	server	Mem: 48 MB	CPU: 0 %		
Custom Metrics				Metadata	
Heap Size	22.19 MiB			App Name	server
Heap Usage	87.38 %			Namespace	default
Used Heap Size	19.39 MiB			Version	0.0.1
Active requests	0			Restarts	0

7. pm2

- pm2 kill로 프로세스 전체 종료 가능

콘솔

```
$ npx pm2 kill && npm start
```

- 재시작하면 프로세스가 CPU 코어 개수만큼 실행됨

▼ 그림 15-3 npx pm2 kill 실행 화면

```
[PM2] [v] Modules Stopped
[PM2] Applying action deleteProcessId on app [all](ids: [ 0 ])
[PM2] [server](0) ✓
[PM2] [v] All Applications Stopped
[PM2] [v] PM2 Daemon Stopped
```

▼ 그림 15-4 npm start 실행 화면

id	name	namespace	version	mode	pid	uptime	↕	status	cpu	mem	user	watching
0	server	default	0.0.1	cluster	42120	2s	0	online	48.4%	54.8mb	zerocho	disabled
1	server	default	0.0.1	cluster	10760	2s	0	online	48.5%	43.1mb	zerocho	disabled
2	server	default	0.0.1	cluster	33784	2s	0	online	54.7%	46.0mb	zerocho	disabled
3	server	default	0.0.1	cluster	42792	2s	0	online	25%	33.9mb	zerocho	disabled
4	server	default	0.0.1	cluster	59596	2s	0	online	9.5%	26.0mb	zerocho	disabled
5	server	default	0.0.1	cluster	26520	1s	0	online	12.4%	25.3mb	zerocho	disabled

8. helmet, hpp로 보안 관리하기

- 모든 취약점을 방어해주진 않지만 실무에서 필수인 패키지

콘솔

```
$ npm i helmet hpp
```

- 배포 환경일 때만 사용하면 됨

app.js

```
...  
const passport = require('passport');  
const helmet = require('helmet');  
const hpp = require('hpp');  
...  
if (process.env.NODE_ENV === 'production') {  
  app.use(morgan('combined'));  
  app.use(helmet());  
  app.use(hpp());  
} else {  
  app.use(morgan('dev'));  
}  
...
```

Contents

- 서비스 운영을 위한 Packages
- [부하 테스트](#)
- 사용자 행동 분석

1. 부하 테스트란?

- 서버가 얼마만큼의 요청을 견딜 수 있는지 테스트
 - 서버가 몇 명의 동시 접속자를 수용할 수 있는지 예측하기 매우 어려움
 - 실제 서비스 중이 아니라 개발 중일 때는 더 어려움
 - 코드에 문제가 없더라도 서버 하드웨어 때문에 서비스가 중단될 수 있음(메모리 부족 문제 등)
 - 부하 테스트를 통해 미리 예측할 수 있음
- Artillery 사용

콘솔

```
$ npm i -D artillery  
$ npm start
```

2. Artillery 사용하기

- 새 콘솔에서 다음 명령어 입력
 - Count 옵션은 가상의 사용자 수
 - N 옵션은 횟수
 - 100명의 사용자가 50번씩 요청을 보내는 상황
- 결과 보고서
 - 사용자 생성(scenarios launched)
 - 테스트 성공(scenarios completed)
 - 요청 성공 횟수(requests completed)
 - 초당 요청 처리 횟수(RPS sent)
 - 응답 지연 속도(Request latency)
 - Min: 최소, Max: 최대, median: 중앙값
 - P95: 하위 95%, P99: 하위 99%
 - 하위는 속도 순서를 말함
 - Median과 P95가 많이 차이 나지 않는 게 좋음

콘솔

```
$ npx artillery quick --count 100 -n 50 http://localhost:8001
(중간 로그 생략)
All virtual users finished
Summary report @ 22:06:27(+0900) 2020-02-26
Scenarios launched: 100
Scenarios completed: 100
Requests completed: 5000
RPS sent: 474.38
Request latency:
  min: 9.1
  max: 303.1
  median: 176.2
  p95: 252.2
  p99: 276.7
Scenario counts:
  0: 100 (100%)
Codes:
  200: 5000
```

3. 여러 페이지 요청 시나리오

- loadtest.json에 사용자의 행동 흐름 작성 가능
 - target: 요청 도메인
 - Phases에서 duration: 몇 초 동안(60초)
 - arrivalRate: 매 초 몇 명(30명)
 - flow: 사용자의 이동
 - get, post 등의 메서드를 나타냄
 - url은 이동한 url
 - json은 서버로 전송한 데이터
- 현재 GET /, POST /auth/login, GET /hashtag 순

loadtest.json

```
{
  "config": {
    "target": "http://localhost:8001",
    "phases": [
      {
        "duration": 60,
        "arrivalRate": 30
      }
    ]
  },
  "scenarios": [
    {
      "flow": [
        {
          "get": {
            "url": "/"
          }
        }, {
          "post": {
            "url": "/auth/login",
            "json": {
              "email": "zerohch0@gmail.com",
              "password": "nodejsbook"
            }
          }
        }, {
          "get": {
            "url": "/hashtag?hashtag=nodebird"
          }
        }
      ]
    }
  ]
}
```

4. 여러 페이지 요청 시나리오(실행)

- 문제점 발견
 - 요청 후반부가 될 수록 응답 시간이 길어짐
 - 첫 응답은 4.7밀리초, 마지막 응답은 51초
 - 5400개 요청은 응답코드 200, 1800개는 응답코드 300
 - 서버가 지금 정도의 요청을 감당하지 못함
 - 서버 사양을 업그레이드하거나, 여러 개 두거나 코드를 더 효율적으로 개선하는 방법 등.
 - 현재는 싱글코어만 사용하므로, 클러스터링 기법 도입 시도
 - arrivalRate를 줄이거나 늘려서 어느 정도 수용 가능한지 체크해보는 것이 좋음
 - 여러 번 테스트하여 평균치를 내보는 게 좋음

콘솔

```
$ npx artillery run loadtest.json
(중간 로그 생략)
All virtual users finished
Summary report @ 23:09:36(+0900) 2020-02-26
  Scenarios launched: 1800
  Scenarios completed: 1800
  Requests completed: 7200
  RPS sent: 50.2
  Request latency:
    min: 4.7
    max: 51806.1
    median: 19150.8
    p95: 45585.6
    p99: 49574.6
  Scenario counts:
    0: 1800 (100%)
  Codes:
    200: 5400
    302: 1800
```

5. 테스트 범위

- 어떤 것을 테스트하고 어떤 것을 테스트 하지 않을지 결정
 - 자신이 짰 코드는 최대한 많이 테스트하기
 - npm을 통해 설치한 패키지는 테스트하지 않음(그걸 만든 사람의 몫임)
 - 우리는 그 패키지/라이브러리를 **사용하는 부분만** 테스트
 - 시스템 테스트: QA처럼 테스트 목록을 두고 체크해나가면서 진행하는 테스트
 - 인수 테스트: 알파 테스트/베타 테스트처럼 특정 사용자 집단이 실제로 테스트
 - 다양한 종류의 테스트를 주기적으로 수행해 서비스를 안정적으로 유지하는 게 좋음

Contents

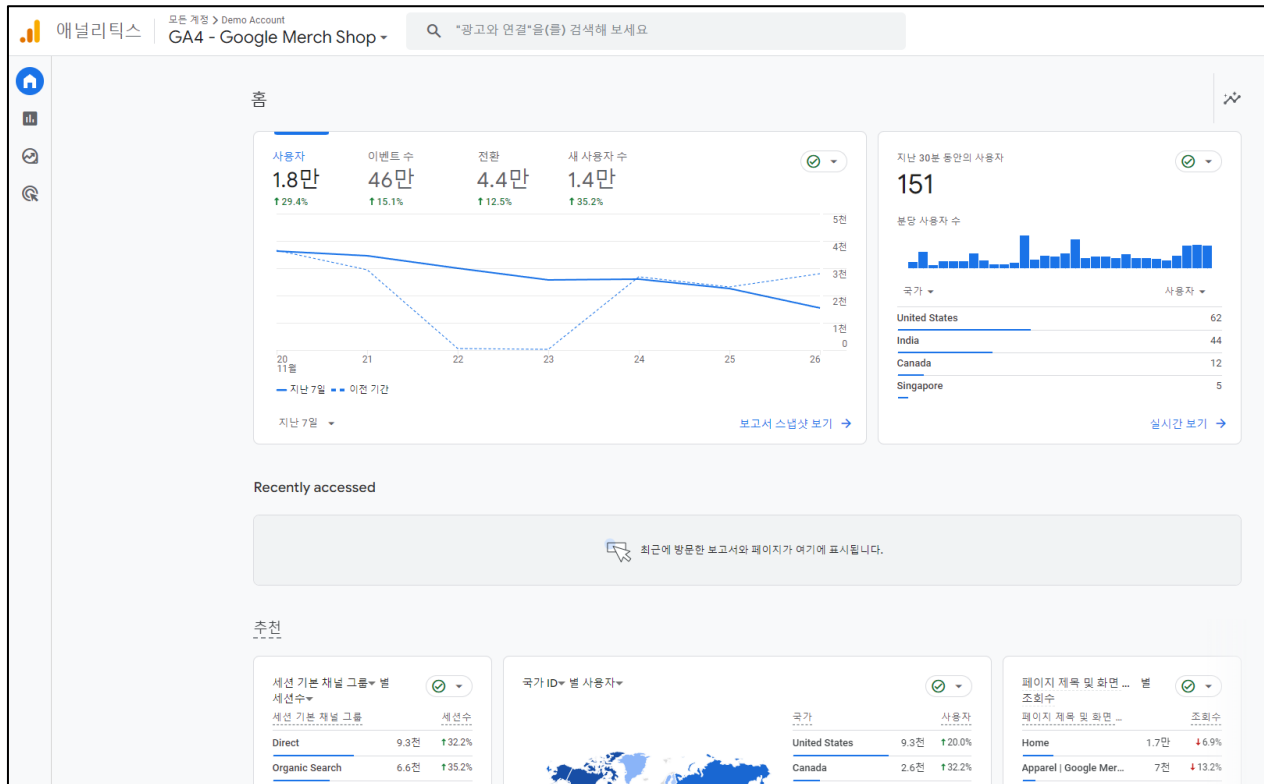
- 서비스 운영을 위한 Packages
- 부하 테스트
- [사용자 행동 분석](#)

Google Analytics

- 운영을 위한 준비 단계
 - 누가 나의 사이트에 들어오는가
 - 어디서부터 타고 들어오는가
 - 어떤 행동을 하는가
- 구글 마케팅 플랫폼 브랜드 내의 플랫폼, 웹사이트 트래픽을 추적하고 보고하는 웹 애널리틱스 서비스
- 학습 아카데미
 - <https://skillshop.exceedlms.com/student/path/1050061-google>

데모 계정

- <https://analytics.google.com/analytics/web>



웹사이트 등록하기

- google analytics 가입
- 새로운 페이지 만들기
- 내가 만든 웹사이트에 추적 코드 삽입

웹사이트 추적

범용 사이트 태그(gtag.js)

이 속성의 범용 사이트 태그(gtag.js) 추적 코드입니다. 이 코드를 복사하여 추적할 모든 웹페이지의 <HEAD>에 첫 번째 항목으로 붙여넣으세요. 이미 페이지에 범용 사이트 태그가 있다면 아래 스니펫의 **config** 행을 기존 범용 사이트 태그에 추가하기만 하면 됩니다.

```

<!-- Global site tag (gtag.js) - Google Analytics -->
<script async src="https://www.googletagmanager.com/gtag/js?id=UA-54516992-1"></script>
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());

  gtag('config', 'UA-54516992-1');
</script>
```

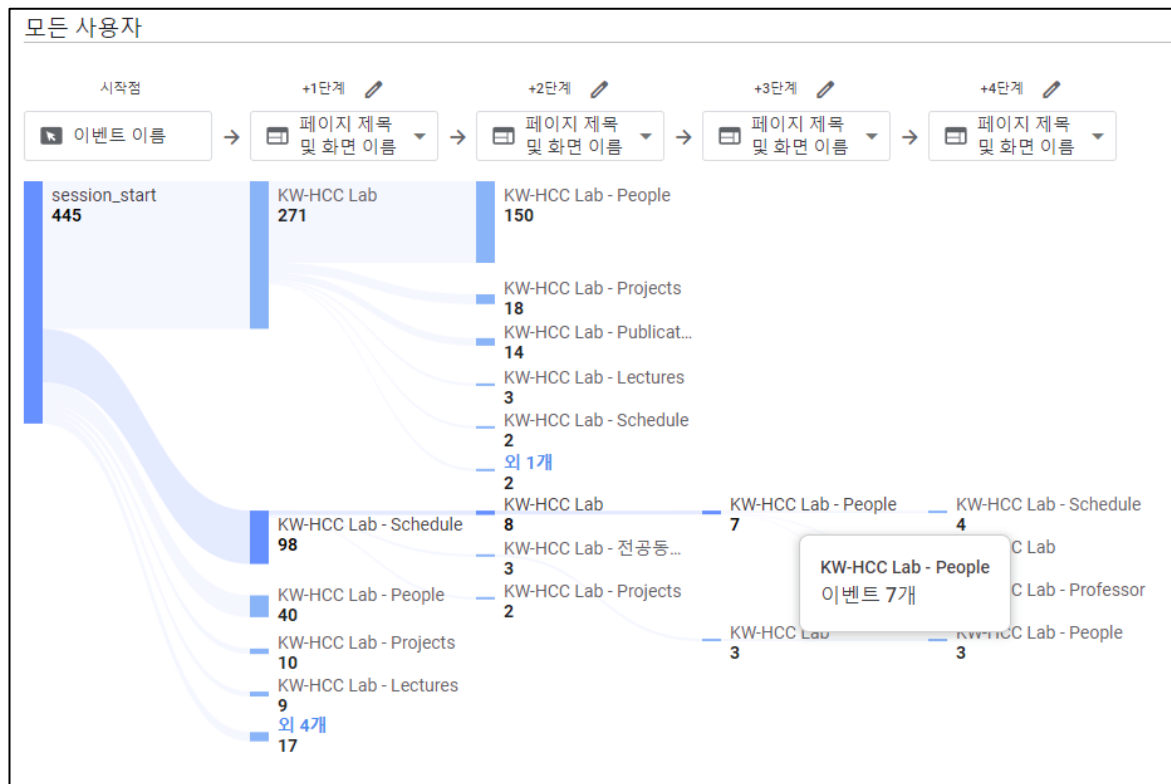
Google Analytics 4

- 가볍게 살펴볼 수 있는 분석 : 페이지뷰 관련

페이지 제목 및 화면 이름	조회수	사용자당 조회수	이탈	↓총 사용자
총계	1,108 총계 대비 100%	4.47 평균과 동일	385 총계 대비 100%	250 총계 대비 100%
1 KW-HCC Lab	364	1.88	125	194
2 KW-HCC Lab - People	271	2.01	75	135
3 KW-HCC Lab - Schedule	157	2.04	107	77
4 KW-HCC Lab - Projects	96	1.92	14	50
5 KW-HCC Lab - Publications	78	1.7	13	46
6 KW-HCC Lab - Professor	49	1.26	16	40
7 KW-HCC Lab - Lectures	51	1.34	20	38
8 KW-HCC Lab - 전공동아리	36	1.24	13	30
9 KW-HCC Lab - Researchers	5	2.5	1	2
10 KW-HCC Lab - Seminars	1	1	1	1

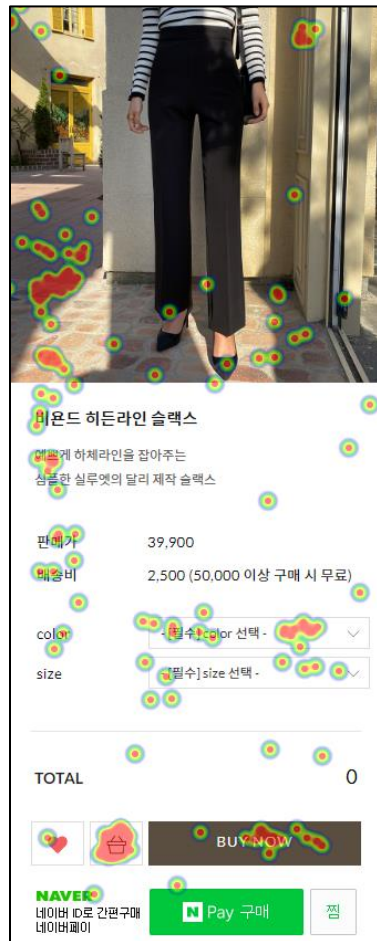
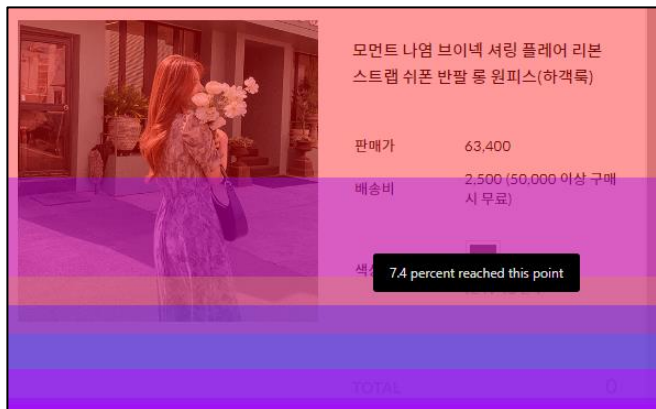
Google Analytics 4

- 가볍게 살펴볼 수 있는 분석 : 경로 탐색 분석



그 밖의 사용자 행동 분석 도구

- hotjar
- amplitude
- beusable
- matomo
 - on-premise



Google Tag Manager (GTM)

- 코드 상 모든 Tag 에 대한 이벤트를 받아올 수 있음
 - google analytics 보다 강력한 기능 제공



Google Tag Manager (GTM)

- 태그 : 수집하고자 하는 관심 이벤트
- 트리거
 - 태그를 실행시키기 위한 조건
 - 페이지뷰, 요소 클릭, 요소 노출, 스크롤 깊이
- 변수
 - 태그에 동적인 정보를 전달
 - 트리거의 실행 조건을 정의

Google Tag Manager (GTM)

- 설정 사례

☐ 이름 ↑	유형	트리거 실행
☐ [HCC] 기본 이벤트	Google 애널리틱스: GA4 구성	 All Pages
☐ [HCC] 전자상거래_장바구니담기	Google 애널리틱스: GA4 이벤트	 [GA4]_EVENT_addToCart
☐ [HCC] 전자상거래_주문완료	Google 애널리틱스: GA4 이벤트	 [GA4]_EVENT_purchase
☐ [HCC]best_exposure	Google 애널리틱스: GA4 이벤트	 [HCC]EVENT_best_exposure
☐ [HCC]best_PUSH_INFO	맞춤 HTML	 [HCC]best_exposure

Google Tag Manager (GTM)

- GA 에서 확인

총계	881,107 총계 대비 100.0%
1 page_view	415,963
2 user_engagement	313,552
3 scroll	55,675
4 session_start	55,138
5 first_visit	36,539
6 view_search_results	1,866
7 click	1,411
8 purchase	671
9 add_to_cart	292

Thank you

Q & A